Global Health TRE - MVP model and architecture

0. Pain-point scope and traceability

Product: Global Health Trusted Research Environment (TRE) **Course:** Advanced Software Engineering - Model and Architect Your MVP **Team:** Anu, Ade, Manushi, Jibek

The MVP vaults a raw upload once, audits it, de-identifies into a new analysis object, and lets Dr. Ezer Kang explore that object under a Project after Daniel Okoro approves. Analysis is spreadsheet/BI-style inside the TRE (columns, rows, charts). SPSS is not required. Soft WORM: write once, read many.

Personas in this MVP

Persona	Role in the MVP
Dr. Ezer Kang	Medical researcher. Creates Projects, requests access, explores and charts inside the TRE, may download only when approved.
Dr. Amina Kamau	Clinical research coordinator. Uploads raw study data and documents local terms.
Daniel Okoro	Data custodian. Approves or denies project-scoped access and can see who asked, why, and what was decided.

Traceability table

ID	Persona	Pain point	MVP capability	Use case(s)	Success signal
PP-1	Dr. Ezer Kang	He depends on SPSS and hand cleanup before he can explore a study file.	After approval he explores the de-identified analysis dataset inside the TRE: select columns and rows, filter, build charts (bar, line, scatter, pie). Prepare includes dictionary and	UC-5 Prepare analysis dataset; UC-8 Explore analysis dataset; UC-9 Create chart	He finishes one explore session and one chart on an approved project dataset without leaving the TRE.

ID	Persona	Pain point	MVP capability	Use case(s)	Success signal
			accepted translations.		
PP-2	Dr. Amina Kamau	Study data sits unused because cleanup takes too long, and uploads are not safely vaulted.	She uploads a CSV/spreadsheet write-once to object storage. An audit row stores user, file name, and timestamp. Originals are not overwritten.	UC-1 Upload dataset; UC-2 Check dataset	She gets a check report in one sitting, and an upload audit row exists for a new object id.
PP-3	Dr. Amina Kamau	Local terms differ by site and get misunderstood.	She marks a local term and writes its definition before the dataset can be prepared.	UC-3 Document local term	A flagged term cannot be prepared without a definition.
PP-4	Daniel Okoro	Sensitive data can be reused with no record of who, why, or benefit, including downloads.	Access is requested per Project with purpose and benefit. He approves or denies. Download is gated and audited.	UC-6 Request project access; UC-7 Review access request; UC-10 Download analysis dataset	He can list project, requester, purpose, benefit, decision, and timestamp. A download requires approval and creates an audit row.
PP-5	All / Safe data	Identifiers in raw uploads make research use unsafe.	Process reads the vaulted original only. Rule-based de-identification writes a new analysis object. The origin object id is unchanged.	UC-4 Process and de-identify dataset	Explore, charts, and download use the de-identified object only.

Reserved use cases

ID	Name	Pain point	Relationship
UC-1	Upload dataset	PP-2	Write-once store + upload audit.
UC-2	Check dataset	PP-2	«include» from UC-1.

ID	Name	Pain point	Relationship
UC-3	Document local term	PP-3	«extend» on UC-1 when a local term appears.
UC-4	Process and de-identify dataset	PP-5	Read-only on original; new analysis object.
UC-5	Prepare analysis dataset	PP-1	Dictionary + accepted translations.
UC-6	Request project access	PP-4	Kang: purpose + benefit for one Project and Dataset.
UC-7	Review access request	PP-4	Custodian approve/deny.
UC-8	Explore analysis dataset	PP-1	Select columns/rows, filter, table view.
UC-9	Create chart	PP-1	Bar, line, scatter, or pie from the current view.
UC-10	Download analysis dataset	PP-4	Gated download + audit.

Out of scope

- Full Google Sheets / Power BI product, notebooks, or arbitrary R/SPSS scripts.
- Strong privacy methods (NLP redaction, k-anonymity, differential privacy).
- Cultural instrument redesign beyond accepted translation.
- Qualitative coding; hospital operations / ministry reporting.
- Automatic approval; hard storage immutability hardware.
- Sharing by email or any path that skips project access approval.

Walkthrough thread

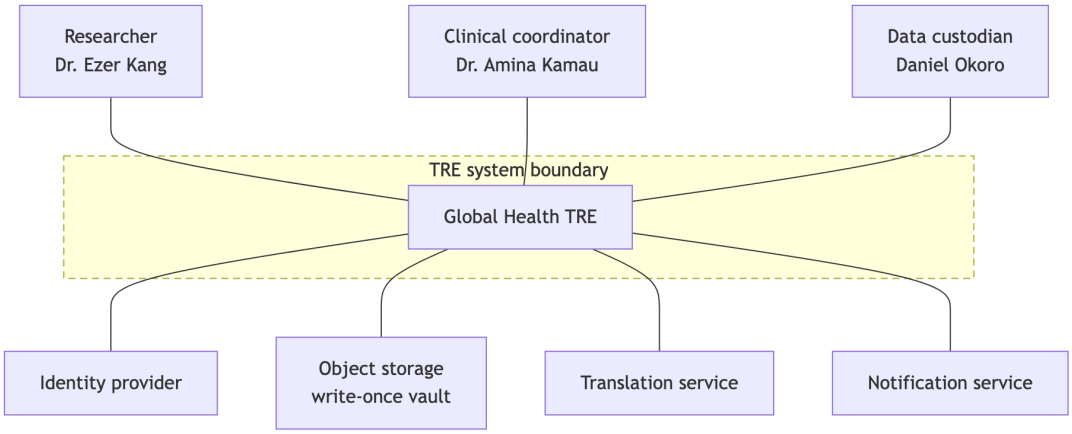
Upload → audit → analyze under a project: UC-1 → UC-4 → UC-6 → UC-7 → UC-8 / UC-9.

Architecture constraint

Modular monolith. Upload path write-only to object storage; process and analysis read-only on vaulted originals. PostgreSQL holds metadata, projects, requests, views, and audit rows.

A1. Context diagram

Caption: This context diagram supports PP-1, PP-2, PP-3, PP-4, and PP-5 by showing the TRE between the three personas and the external systems it uses.



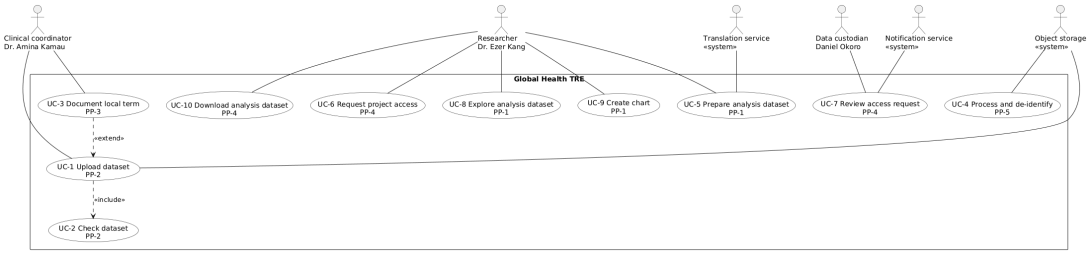
Context diagram

Boundary justification

The TRE builds upload audit, project-scoped access, de-identification into a new analysis object, spreadsheet-style explore, charts, and gated download records. Sign-in, the vault, machine translation, and notification delivery are integrated. We chose to build the approval and audit rules; we chose to integrate the identity provider instead of storing passwords ourselves.

A2. Use case diagram

Caption: This use case diagram supports PP-1, PP-2, PP-3, PP-4, and PP-5. Each use case is tagged with its PP-ID.



Use case diagram

PlantUML source: `A2-use-cases.puml`.

Three human actors and three system actors. **«include»:** UC-1 → UC-2 (check always runs). **«extend»:** UC-3 → UC-1 (local term only when present). Ten use cases (within 6–10), each tagged with a PP-ID.

A3. Textual use cases

The two use cases that matter most to the MVP walkthrough: **UC-1 Upload dataset** (PP-2) and **UC-8 Explore analysis dataset** (PP-1).

UC-1 Upload dataset

Caption: This use case supports PP-2. It writes the file once and records who uploaded it.

Row	Detail
Actors	Clinical coordinator (Dr. Amina Kamau). Identity provider. Object storage. TRE through Dataset and AuditRecord.
Description	The coordinator uploads a spreadsheet or CSV she already has. The TRE stores the bytes write-once in object storage and writes an upload audit row with user, file name, and timestamp.
Data	session, fileName, file bytes, userId, timestamp.
Stimulus	The coordinator submits the upload form.
Response	The TRE returns datasetId and objectId. Status becomes Uploaded. AuditRecord stores action uploaded.
Pre/post-conditions	Pre: coordinator has a session. Post: a new object exists in storage and was not an overwrite of an existing objectId. An upload audit row exists.
Comments	Soft WORM starts here. Check dataset is included and always runs after a successful store.
Alternative flows	1. Identity verify fails: upload rejected, no object written. 2. Storage rejects overwrite (objectId already exists): upload rejected. 3. Check finds blocking shape errors: dataset becomes CheckFailed and is not processed.

UC-8 Explore analysis dataset

Caption: This use case supports PP-1. Spreadsheet-style explore after project approval.

Row	Detail
Actors	Researcher (Dr. Ezer Kang). Identity provider. Notification service. TRE through Project, AccessRequest, AnalysisView, Dataset, AuditRecord.
Description	After an approved project request, the researcher opens the de-identified analysis dataset, selects

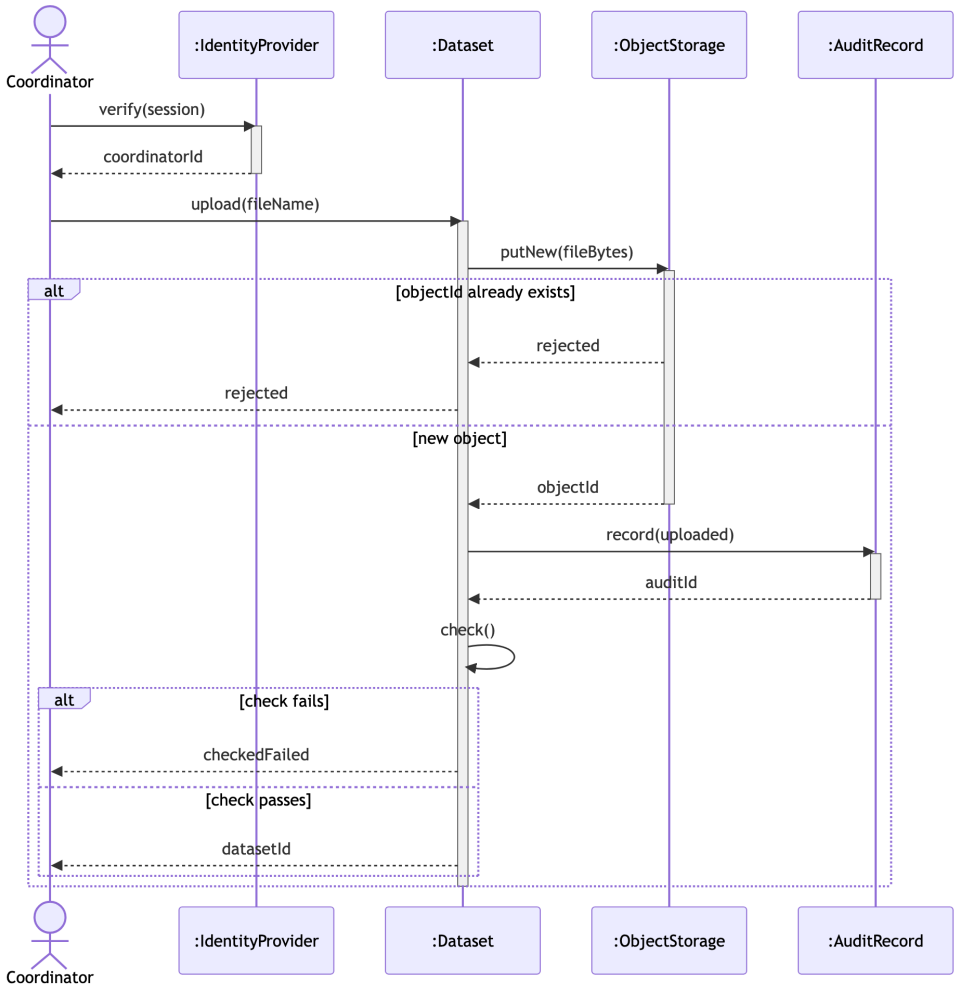
Row	Detail
	columns and rows, applies filters, and sees a table view inside the TRE.
Data	session, projectId, datasetId, selectedColumns, rowFilters, viewId.
Stimulus	The researcher opens Explore on an approved project dataset and applies a column or row selection.
Response	The TRE returns an AnalysisView. An audit row stores action explored. The researcher is notified asynchronously that the view is ready.
Pre/post-conditions	Pre: researcher has a session; Project exists; AccessRequest for that project and dataset is Approved; Dataset is AnalysisReady. Post: an AnalysisView is stored under the Project; original vault object is unchanged.
Comments	This replaces SPSS for ordinary exploration. UC-9 Create chart builds on the current AnalysisView.
Alternative flows	1. No approved AccessRequest: explore rejected. 2. Dataset not AnalysisReady: explore rejected. 3. Selected column still marked identifier after de-id: view builder rejects that column.

A4. Sequence diagrams

One sequence per textual use case in A3. Lifelines use `:ClassName`. Filled arrowhead = synchronous. Open arrowhead = asynchronous. Dashed = return.

UC-1 Upload dataset

Caption: This sequence supports PP-2: verify user, write-once store, upload audit, then included check.

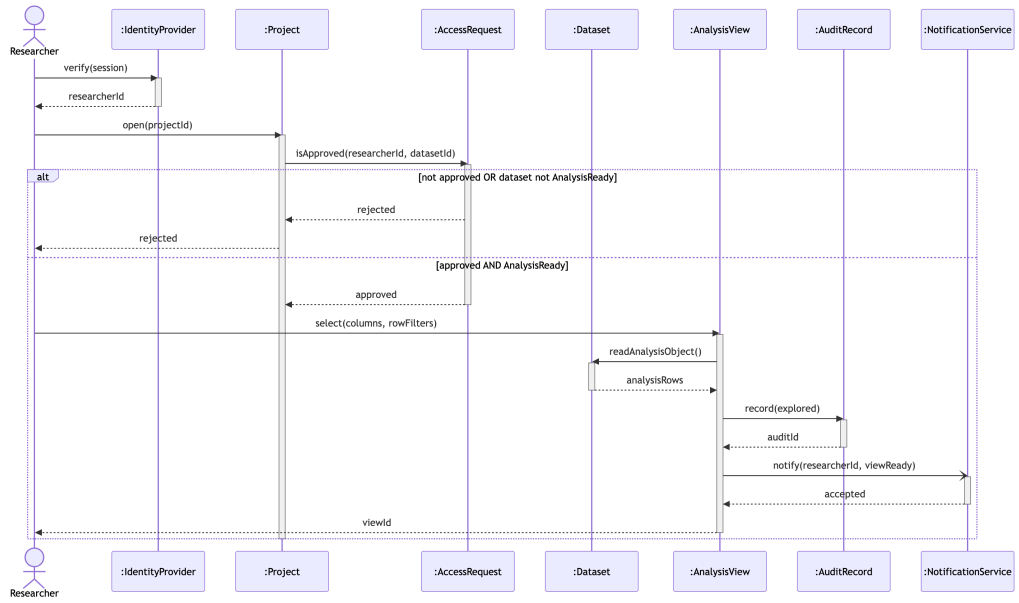


Sequence for UC-1

External system: `:ObjectStorage` and `:IdentityProvider` from the context diagram. Guards are in brackets inside `alt`.

UC-8 Explore analysis dataset

Caption: This sequence supports PP-1 after project approval: verify, confirm access, build a column/row view, audit, async notify.

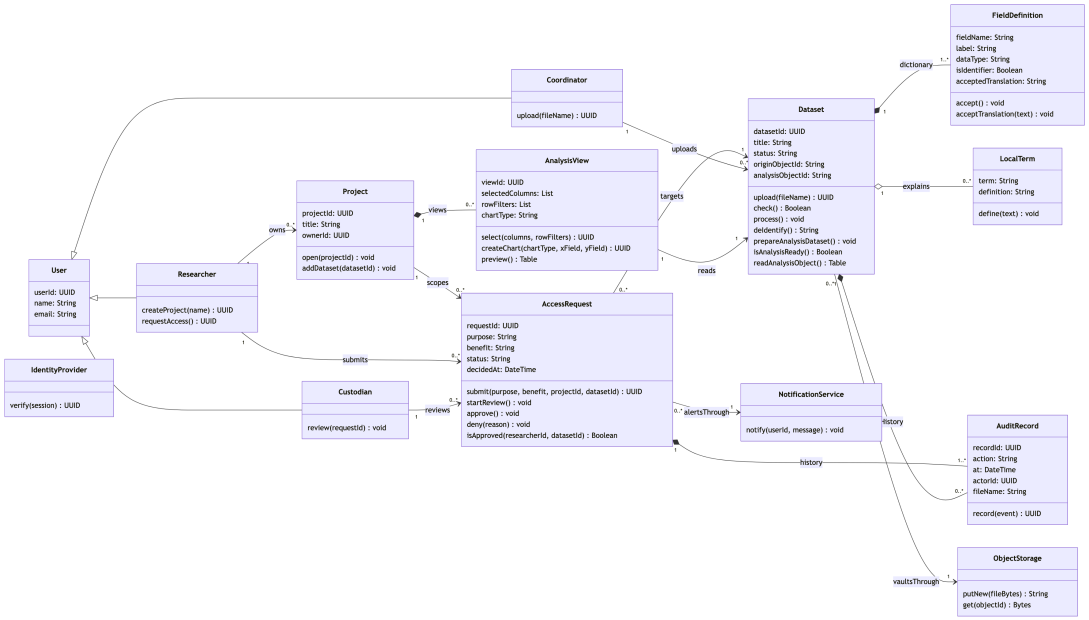


Sequence for UC-8

`notify` uses an open arrowhead (asynchronous). Returns are dashed. Explore never writes to object storage.

A5. Domain class diagram

Caption: This class diagram supports PP-1, PP-2, PP-3, PP-4, and PP-5. Eleven domain classes (within 6–12) plus three external system classes required by A4.



Class diagram

Domain classes (11): User, Researcher, Coordinator, Custodian, Project, Dataset, FieldDefinition, LocalTerm, AccessRequest, AnalysisView, AuditRecord.
External system classes (for A4): IdentityProvider, ObjectStorage, NotificationService.

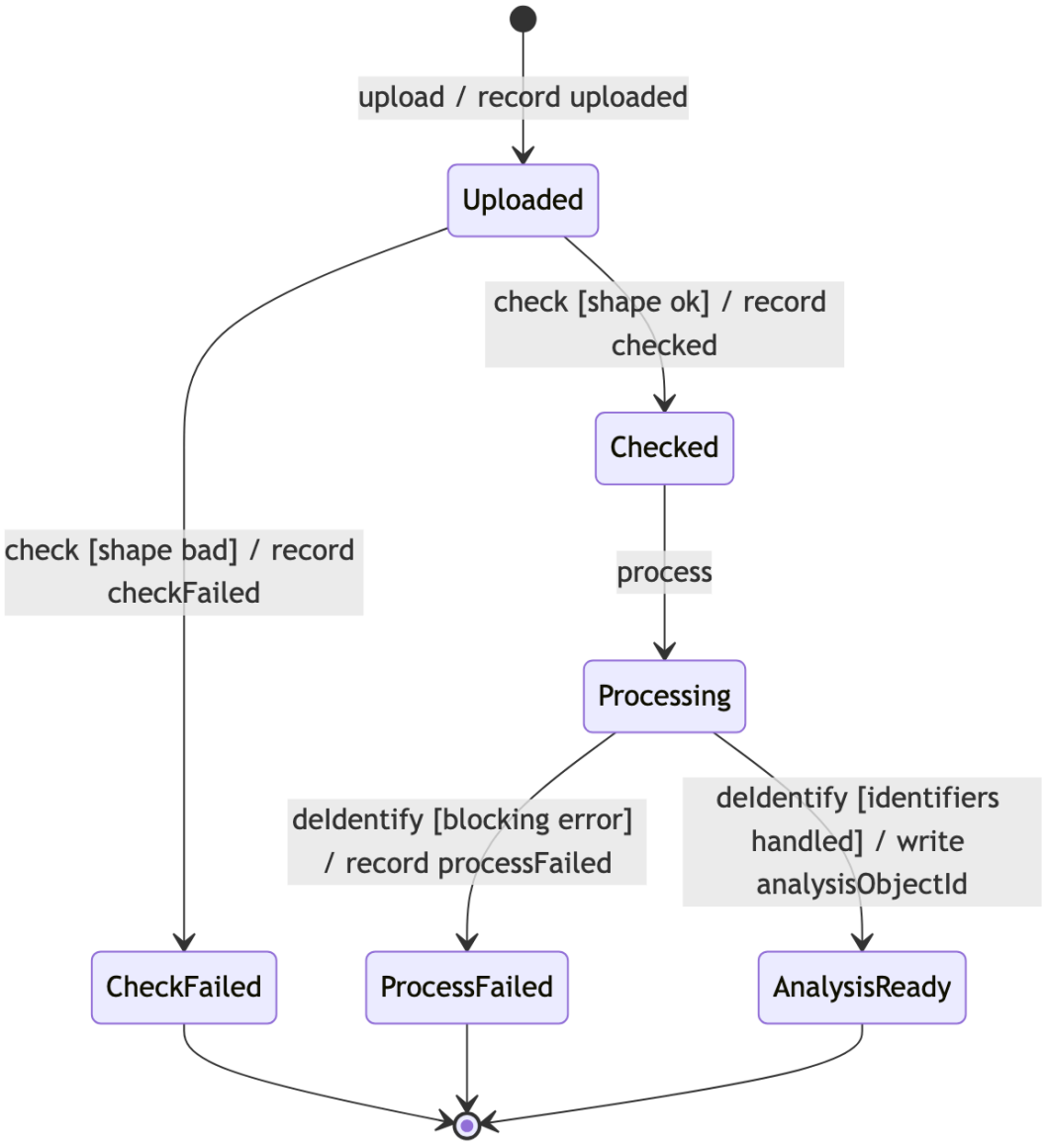
Generalization: Researcher, Coordinator, Custodian specialize User.
Composition: FieldDefinition dictionary and AuditRecord history die with their whole.
Aggregation: LocalTerm can be reused across datasets from the same site.

Soft WORM: `ObjectStorage.putNew` only; Dataset keeps separate `originObjectId` and `analysisObjectId`. Charts use `AnalysisView.createChart`.

Every A4 message is an operation on the receiving class: `verify`, `upload`, `putNew`, `record`, `check`, `open`, `isApproved`, `select`, `readAnalysisObject`, `notify`.

A6. State machine diagram

Caption: This state machine supports PP-2 and PP-5. The entity is Dataset, from write-once upload through de-identified AnalysisReady.



Dataset state machine

Six states: Uploaded, Checked, CheckFailed, Processing, AnalysisReady, ProcessFailed. Initial and final pseudostates included. Failure paths: CheckFailed and ProcessFailed. Guards and actions appear on check and deIdentify.

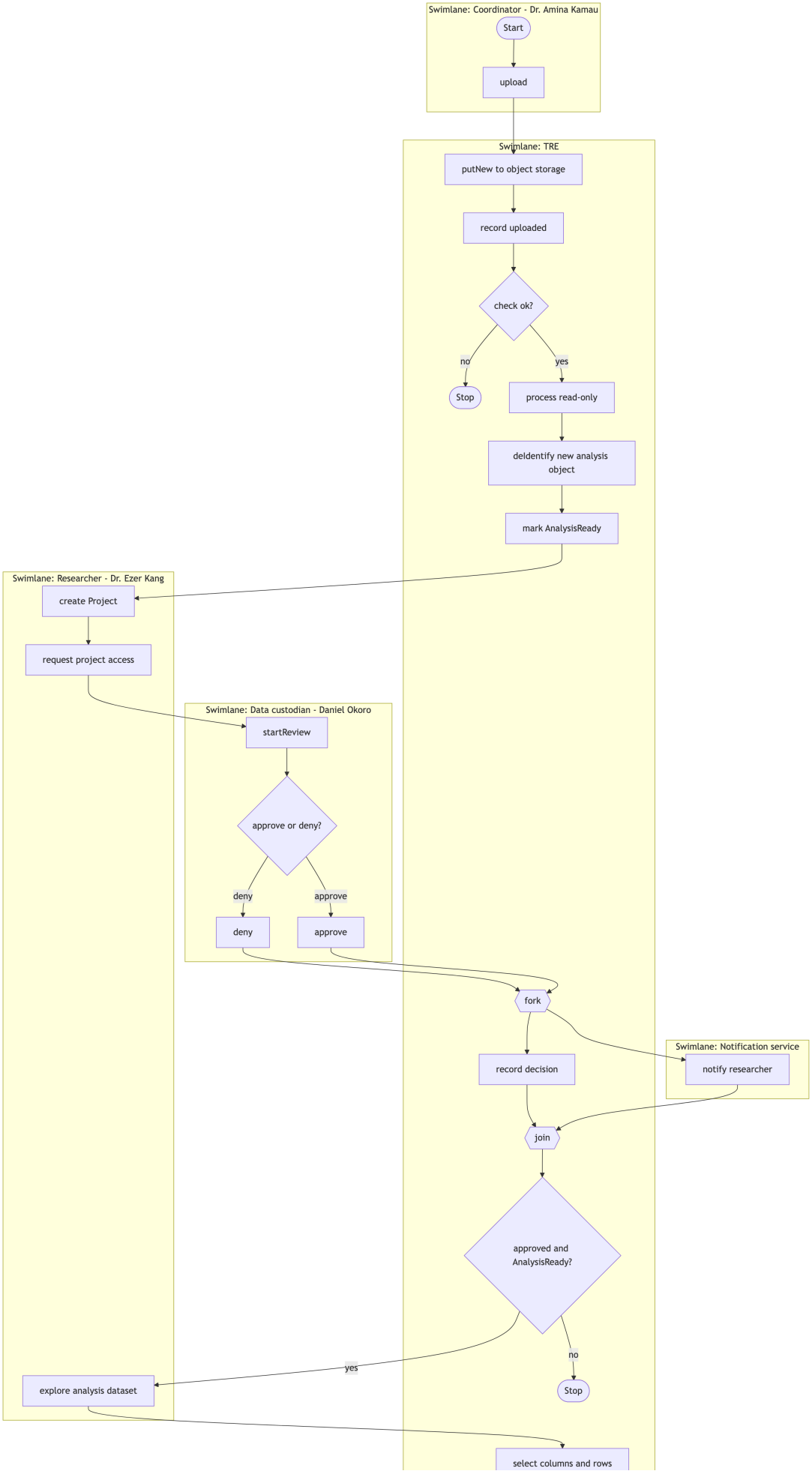
There is no transition that overwrites `originObjectId`. AnalysisReady writes a **new** `analysisObjectId`.

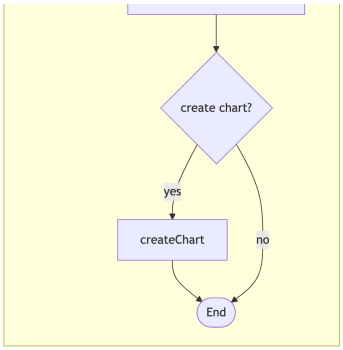
Event	Also appears in
upload	A4 UC-1, A7
check	A4 UC-1, A7
process	A7
deIdentify	A7

Explore requires AnalysisReady and an approved AccessRequest (`isApproved` in A4 UC-8).

A7. Activity diagram

Caption: This activity supports the walkthrough upload → audit → analyze under a project (PP-2, PP-5, PP-4, PP-1).


Swimlane: Coordinator - Dr. Amina Kamau
Start
upload
Swimlane: TRE
putNew to object storage
record uploaded
check ok?
no
yes
Stop
process read-only
deIdentify new analysis object
mark AnalysisReady
Swimlane: Researcher - Dr. Ezer Kang
create Project
request project access
Swimlane: Data custodian - Daniel Okoro
startReview
approve or deny?
deny
approve
deny
approve
fork
record decision
Swimlane: Notification service
notify researcher
join
approved and AnalysisReady?
yes
no
Stop
explore analysis dataset
select columns and rows



Activity diagram

Swimlanes map to A1 actors/systems. Decisions use guards. After the custodian decides, recording the decision and notifying the researcher run in parallel (fork/join). Explore and optional chart run only when access is approved and the dataset is AnalysisReady.

B1. Architecture decision record

Caption: This decision supports PP-1, PP-2, PP-3, PP-4, and PP-5. Drivers are soft WORM upload audit, project-scoped approval, rule-based de-identification, and in-TRE explore/charts.

1. Context

Four people build the first version. Users: researcher (creates Projects), clinical coordinator (uploads), custodian (approves). Load is class-project scale.

Sensitive health data must be auditable. Upload must write once and leave an audit row (PP-2). Process must read the original and write a new de-identified analysis object (PP-5). After approval (PP-4), Kang explores columns/rows and builds charts inside the TRE (PP-1). Download stays gated.

2. Decision

Modular monolith. One application, one deployable, one database.

Module	Use cases	Owns
Intake	UC-1, UC-2, UC-3	Upload, check, local terms, upload audit
Process	UC-4	Read-only de-identify, new analysis object
Dictionary / prepare	UC-5	FieldDefinition, translations
Project access	UC-6, UC-7, UC-10	Project, AccessRequest, gated download
Analysis	UC-8, UC-9	AnalysisView, charts

Modules call each other in process. Kubernetes deploys this one application.

3. Alternatives considered

Microservices. Rejected. Approval, project membership, and AnalysisReady must agree in one transaction. Four students should not operate five images.

Event-driven core. Rejected. Upload and approval are form waits. Notification is one async HTTPS call, not a broker for every step.

Serverless only. Rejected. Process and chart rendering outlive a short function. One long-running app matches the class diagram.

4. Performance consequences

Upload latency: object-storage putNew + one audit insert. Process/de-identify is slower and off the custodian click path. Explore/charts read the analysis object and never rewrite the origin. Scale by replicas of the same image. Cache AnalysisReady until process completes. Chart render is synchronous for MVP size.

5. Security consequences

Trust boundaries: user network, TRE application, health-data stores, external partners (see B2).

Authentication: `IdentityProvider.verify(session)` (OIDC). No local passwords. Authorization: Coordinator uploads; Researcher creates projects, explores, charts, requests download; only Custodian approves.

Soft WORM: `ObjectStorage.putNew` only; process/analysis use `get`. Secrets for DB and translation API come from Kubernetes Secrets (`tre-db`, `tre-api`). Gated download checks `AccessRequest.isApproved` and writes `AuditRecord`.

6. Cloud-agnostic plan

Capability	Open standard	Lock-in accepted
Run app	Kubernetes	Managed control plane only
Sign-in	OpenID Connect	Any OIDC IdP
Files	S3-compatible API	putNew/get only
Records	SQL / PostgreSQL wire	Ordinary SQL
Translation	HTTPS JSON	Provider swappable; accepted text stored by us

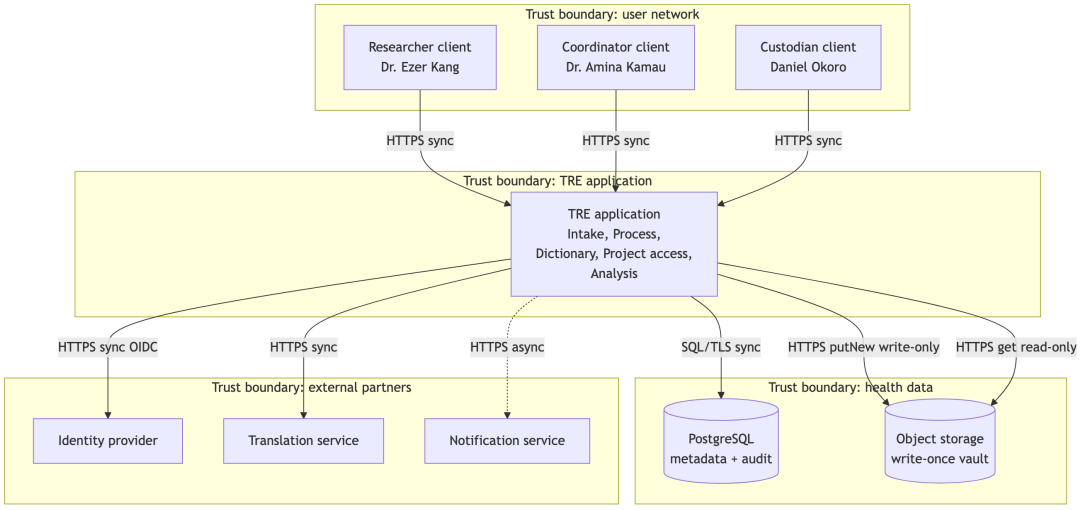
Capability	Open standard	Lock-in accepted
Notification	HTTPS JSON	Delivery external; audit does not depend on it

Approval and WORM rules stay in modules we build.

B2. Architecture diagram

Caption: This architecture supports PP-1, PP-2, PP-3, PP-4, and PP-5. Solid arrows are synchronous; the dotted arrow is async notification.

Legend: box = client, service, or external system. Cylinder = data store. Dashed box = trust boundary.



Architecture diagram

Modules match A5 clusters. External systems match A1. Every trust-boundary crossing has a protocol label. Authentication for the OIDC arrow and authorization for roles are in B1 section 5.

C. Kubernetes manifest

Caption: This manifest supports PP-1 through PP-5 by deploying the one TRE application from B2 (intake, process, project access, analysis).

A Kubernetes manifest is YAML that states the desired setup: image, replica count, CPU and memory, and how traffic reaches the pods. This project points at `tre-team/app:0.1.0`. Files are in `deploy/tre-app.yaml`. `kubeconform` validates them offline.

How it scales and how it is secured

Deployment starts at 2 replicas. HorizontalPodAutoscaler grows to 6 when average CPU stays above 70 percent. Service type is ClusterIP. NetworkPolicy allows inbound 8080 only, and outbound DNS, PostgreSQL 5432, and HTTPS 443 for identity, object storage, translation, and notification. Secrets `tre-db` and `tre-api` supply the database password and translation key. `runAsNonRoot: true` keeps the process off root. Soft WORM is enforced in the application; the NetworkPolicy still limits which ports the pods may use to reach the vault over 443.

```
$ kubeconform -verbose -summary -strict deploy/*.yaml
deploy/tre-app.yaml - Service tre-app is valid
deploy/tre-app.yaml - NetworkPolicy tre-app is valid
deploy/tre-app.yaml - HorizontalPodAutoscaler tre-app is valid
deploy/tre-app.yaml - Deployment tre-app is valid
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
exit code: 0
```

kubeconform result

Contribution statement

Member	Lead deliverables
Anu	A1 Context diagram, A2 Use case diagram, scope table
Ade	A3 Textual use cases, A4 Sequence diagrams, B1 ADR
Manushi	A5 Class diagram, A6 State machine
Jibek	A7 Activity diagram, B2 Architecture diagram, C Kubernetes manifests

Every member authors at least one diagram.